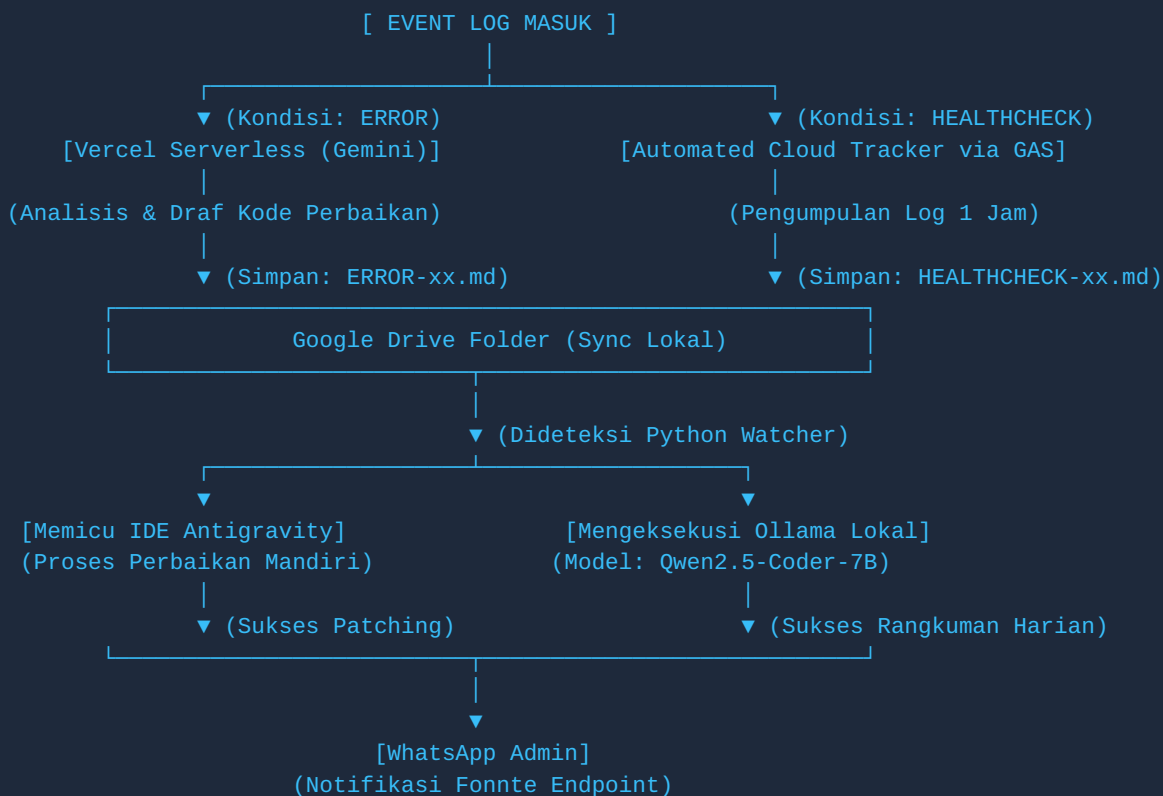


Cetak Biru Arsitektur: Hybrid Self-Healing & Automation AI

Siklus Operasional 24 Jam Terpisah: Gemini 3.5 Cloud (Error Mitigation) & Qwen2.5 Local (Healthcheck Routine)

Dokumen ini menyajikan rencana pembaruan komprehensif (v3) arsitektur loop tertutup pemantauan sistem *SapaTamu* menggunakan pendekatan **Hybrid AI**. Strategi ini memisahkan beban kerja kritis perbaikan kode ke model cloud, dan penanganan rekam rutin ke model AI lokal guna menghemat token, kuota, serta menjamin privasi data log.

1. Topologi Aliran Data Kerja Terpisah (Hybrid Workflow)



2. Jadwal Rutinitas Kerja 24 Jam Agen AI

Jam 07.00 WIB — Startup Perangkat & Aktivasi Watcher

PC Admin menyala otomatis via BIOS Wake-on-LAN atau Task Scheduler Windows. Skrip latar belakang langsung berjalan, membuka ****IDE Antigravity****, dan mulai mengawasi direktori Google Drive lokal.

Event-Driven — Hot-Fix Patching (Prefix: ERROR)

Setiap ada file berawalan `ERROR-` masuk, skrip memerintahkan IDE Antigravity untuk menjalankan siklus otomatisasi perbaikan internal menggunakan Gemini Cloud, melakukan **debugging**, dan melaporkan sukses balik ke WA admin.

Setiap 1 Jam Sekali — Laporan Berkala (Prefix: HEALTHCHECK)

Setiap 1 jam, file berawalan `HEALTHCHECK-` yang berisi aktivitas transaksi CRUD/log mentah akan turun ke Drive. Skrip lokal memprosesnya menggunakan ****AI Lokal (Qwen2.5-Coder via Ollama)**** untuk dirangkum menjadi laporan bahasa alami ke WhatsApp.

Jam 23.59 WIB — Resume Total & Shutdown Aman

Agen AI lokal merangkum statistik total harian, mengirimkannya ke WhatsApp Admin via Fonnte, lalu mengeksekusi perintah `shutdown /s /t 60` untuk mematikan daya PC secara otomatis.

3. Implementasi Skrip Watcher Utama (Python - Lokal Computer)

Gunakan implementasi skrip **Hybrid** di bawah ini pada mesin lokal Anda untuk mengelola pembagian alur eksekusi:

```

import os
import time
import datetime
import requests
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

FOLDER_PATH = "G:/My Drive/Log Perbaikan"
FONNTE_TOKEN = "TOKEN_FONNTE_ANDA"
ADMIN_PHONE = "NOMOR_WHATSAPP_ADMIN"
OLLAMA_URL = "http://localhost:11434/api/generate"

def kirim_whatsapp(pesan):
    url = "https://api.fonnte.com/send"
    payload = {'target': ADMIN_PHONE, 'message': pesan, 'countryCode': '62'}
    headers = {'Authorization': FONNTE_TOKEN}
    try:
        requests.post(url, data=payload, headers=headers)
    except Exception as e:
        print("Gagal kirim WA:", e)

def jalankan_auto_healing_antigravity(file_path):
    print("🔥 Memicu Otomatisasi Perbaikan di Antigravity IDE...")
    time.sleep(5) # Proses patching & debugging internal IDE
    nama_file = os.path.basename(file_path)
    pesan = f"✅ *HOT-FIX SELESAI (ANTIGRAVITY)*\n\nSistem mendeteksi file `{nama_file}`. IDE Anti
    kirim_whatsapp(pesan)

def jalankan_rekap_healthcheck_lokal(file_path):
    print("🕒 Memproses rangkuman log via AI Lokal (Qwen2.5-Coder)...")
    with open(file_path, 'r', encoding='utf-8') as f:
        isi_log_mentah = f.read()

    prompt_instruksi = (
        f"Rangkum aktivitas log mentah 1 jam terakhir ini menjadi pesan WhatsApp pendek, "
        f"padat, dan informatif untuk Admin. Laporkan jika ada aktivitas CRUD pengguna.\n\n"
        f"Log Mentah:\n{isi_log_mentah}"
    )

    payload = {"model": "qwen2.5-coder:7b", "prompt": prompt_instruksi, "stream": False}
    try:
        response = requests.post(OLLAMA_URL, json=payload)
        hasil_ai = response.json().get("response", "Gagal merangkum secara lokal.")
        kirim_whatsapp(f"🕒 *LAPORAN 1 JAM (AI LOKAL)*\n\n{hasil_ai}")
    except Exception as e:
        print("Koneksi ke Ollama gagal:", e)

class LogMovementHandler(FileSystemEventHandler):
    def on_created(self, event):
        if event.is_directory or not event.src_path.endswith('.md'):
            return
        nama_file = os.path.basename(event.src_path)
        if "ERROR" in nama_file:
            jalankan_auto_healing_antigravity(event.src_path)
        elif "HEALTHCHECK" in nama_file:
            jalankan_rekap_healthcheck_lokal(event.src_path)

```

```

if __name__ == "__main__":
    print("Watcher Hybrid AI SapaTamu Aktif di Latar Belakang...")
    event_handler = LogMovementHandler()
    observer = Observer()
    observer.schedule(event_handler, path=FOLDER_PATH, recursive=False)
    observer.start()

    try:
        while True:
            time.sleep(1)
            sekarang = datetime.datetime.now()
            if sekarang.hour == 23 and sekarang.minute == 59:
                kirim_whatsapp("📊 *SYSTEM REPORT*: Siklus kerja harian selesai. PC Admin akan oto
                time.sleep(5)
                os.system("shutdown /s /t 60")
                break
    except KeyboardInterrupt:
        observer.stop()
    observer.join()

```

4. Rekomendasi Ekosistem & Alat Pembantu AI Lokal

Untuk mengelola, memantau, dan melakukan pengujian *prompt* terisolasi pada AI Lokal Anda (Ollama), disarankan menggunakan alat berikut:

- **Page Assist Extension:** Antarmuka obrolan (*chat interface*) berbasis ekstensi browser (Chrome/Edge) yang sangat ringan. Langsung terhubung ke `localhost:11434` tanpa membebani memori RAM komputer Anda.
- **Open WebUI:** Jika Anda membutuhkan dasbor lengkap seperti ChatGPT komersial untuk kebutuhan manajemen model, pengunggahan berkas dokumentasi proyek, dan pembuatan agen kustom lokal.